40 Days of LeetCode Challenge

40 days, 40 problems
DAY :11

LEETCODE PROBLEM NO. 1 : Two Sum

Problem Link : https://leetcode.com/problems/running-sum-of-1d-array/
Arrays · HashMap · key-value pairs · complement search · time-space complexity.

STEP BY STEP SOLUTION

Step 1. Understand the question :-

We're given:

- An integer array nums.
- An integer target.

Our task is to find two different elements in the array whose sum equals the target. Return their indices, not their values. Important conditions:

- Exactly one valid solution exists.
- We cannot use the same array element twice.
- We can return the indices in any order.

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation:
The sum of nums[0] and nums[1] is 2 + 7 = 9, which equals the target. Therefore, the indices [0, 1] are returned.

Example 2:
Input: nums = [3,2,4], target = 6
Output: [1,2]
Explanation:
The sum of nums[1] and nums[2] is 2 + 4 = 6, which equals the target. Therefore, the indices [1, 2] are returned.

Example 3:
Input: nums = [3,3], target = 6
Output: [0,1]
Explanation:
The sum of nums[0] and nums[1] is 3 + 3 = 6, which equals the target. Therefore, the indices [0, 1] are returned.

Step 3. Identify the pattern

The important concept here is Complement Search. Instead of checking every possible pair, we can ask:

complement = target - currentNumber

For example, if the target is 9 and our current number is 2:- 9-2=7

We need to find whether 7 already exists among the previously visited elements.

This is where a Java HashMap becomes useful.

A HashMap stores key-value pairs. Here, we'll store:

- Key: number from the array.
- Value: index of that number.

This allows us to look up the required complement efficiently.

Step 4. Build the logic

Here's the approach we'll follow.

Algorithm

Step 1: Start.

Step 2: Create an empty HashMap named map to store array elements as keys and their corresponding indices as val ues.

Step 3: Traverse the array using a loop from index 0 to nums.length - 1.

Step 4: For each element, calculate its complement:

complement = target - nums[i]

Step 5: Check whether the complement already exists in the HashMap.

- If it exists, retrieve its index using map.get(complement).
- Return a new array containing the complement's index and the current index: {map.get(complement), i}.

Step 6: If the complement does not exist, insert the current element and its index into the HashMap using:

map.put(nums[i], i)

Step 7: Repeat Steps 4–6 until the array is traversed or a valid pair is found.

Step 8: If no pair is found, return an empty array.

Step 9: Stop.

Pseudocode

```
START

  Create an empty HashMap named map

  FOR i = 0 to length of nums - 1

    complement = target - nums[i]

    IF map contains complement THEN

      RETURN [map.get(complement), i]
```

```
        END IF
        Insert nums[i] and its index i into map
    END FOR
    RETURN empty array
STOP
```

Step 5. Code in Java

Approach A

This is the standard arithmetic solution for LeetCode 1.

```
import java.util.HashMap;
class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i}; }
            map.put(nums[i], i);
        } return new int[]{};
    }
}
```

NOTE:-

Approach B – Brute Force (Nested loops)

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        } return new int[]{};
    }
}
```

How it works:

1. The outer loop selects the first number.
2. The inner loop checks every number after it.
3. If their sum equals the target, return their indices.
4. Otherwise, continue checking pairs.
5.

For [2, 7, 11, 15], target 9:

- 2 + 7 = 9 → found!
- Return [0, 1].

Complexity:

- Time: (O(n^2)), because we may check every pair.
- Auxiliary space: (O(1)).
- This approach is easy to understand, but it becomes inefficient for large arrays.

How to submit

1. Open LeetCode Problem 1. Two Sum
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. *import java.util.HashMap;*

Imports Java's HashMap class so we can store number-index pairs.

B. *class Solution {*

Defines the class required by LeetCode.

C. *public int[] twoSum(int[] nums, int target) {*

The method accepts an integer array and target, and returns an integer array containing the two indices.

D. *HashMap<Integer, Integer> map = new HashMap<>();*

Creates an empty HashMap. Both key and value are integers:

Key	Value
Number	Index

For example, if we store the number 2 from index 0, the map contains: {2 = 0}

E. *for (int i = 0; i < nums.length; i++) {*

int complement = target - nums[i];

The loop visits every element. We calculate the complement by subtracting the current number from the target. For example:

- Target = 9
- Current number = 2
- Complement = 9 - 2 = 7

F. *if (map.containsKey(complement)) {*
return new int[]{map.get(complement), i};
}

- map.containsKey(complement) checks whether the required number exists in the HashMap.
- map.get(complement) retrieves its previously stored index.
- i is the index of the current number.
- new int[]{...} creates and returns the answer array.
- For example, if the map contains {2=0} and our current number is 7 at index 1
- we return: [0, 1]

G. *map.put(nums[i], i);*

If the complement isn't found, we store the current number and its index. For example:

map.put(2, 0);

This means the number 2 is stored with index 0.

H. *return new int[]{};*

A fallback empty array. The problem guarantees exactly one solution, so this line won't be reached for valid inputs.

Complete dry run

Let's trace Example 1:

nums = [2, 7, 11, 15]
target = 9
Initially: map = {}

Iteration 1 i = 0

- Current number = 2
- Complement = 9 - 2 = 7
- Is 7 in the map? No.
- Store 2 → 0.
- Map: {2=0}

Iteration 2 i = 1 · Found!

- Current number = 7
- Complement = 9 - 7 = 2
- Is 2 in the map? Yes.
- Retrieve its index: 0.
- Current index: 1.
- RETURNED INDICES :- [0, 1]

Step 7. Time complexity & Auxiliary space complexity

Approach	Time complexity	Auxiliary space
Brute force	(O(n^2))	(O(1))
HashMap	(O(n)) average	(O(n))

NOTE:-

Why is HashMap faster?

The brute-force approach repeatedly checks pairs. The HashMap approach uses average constant-time lookups and insertions, allowing us to process the array in one traversal.

Can we optimize it further?

For a general unsorted array, the HashMap approach is an excellent solution with (O(n)) average time.

We shouldn't claim it is (O(1)) space just because it uses one map in the worst case, the map can store almost all n elements. One more thing sorting and two pointers are possible alternatives, but sorting can change the original indices, so we need to preserve the original index information. The HashMap solution is simpler for this particular problem.

SCREENSHOT:-

